

First eight posts

LC-STRATEGY-V4.0.0 · Selected strategy. Complete proposed copy; exact assets await approval.
Sunil examples are illustrative until product-story evidence is cleared.

Roadmap and launch gates

Audience: Experienced engineer or technical lead · Practical question

Caption

The roadmap has a launch date. Does it have a reason to believe the system will be ready?

Imagine reviewing an agent that prepares a daily plan. The demo works. The team can explain the architecture. The calendar says it is time for a pilot.

Then you ask what happens when the inputs are incomplete. Or when the tool returns something the system cannot use. The answers are still being worked out.

That is the conversation I would want before agreeing to the date.

A date helps people coordinate. A release gate explains what must be true: which behaviour has been tested, what evidence is available, who owns the decision and what happens if the condition is missed.

I would put both on the roadmap. I would also name what we are deliberately leaving out. Otherwise, an extra week can become an extra feature instead of time to resolve the original uncertainty.

For this illustrative system, the next useful milestone might be a bounded trial with a review owner, rather than a wider release.

Look at one milestone on your own roadmap. Could someone outside the team explain what evidence would close it?

Explain audience and programme outcome

Audience: Experienced engineer or technical lead · Explore programme

Caption

You can understand the components of an agent and still have difficult questions about its design.

What is it allowed to do? How will you evaluate it? Which failure should stop a release? How would you explain the trade-off to your team?

At The Living Craft, I bring those questions into practical work. Members build a working agentic system, explain the architecture behind it and revise their work through continuous feedback.

The open cohort is for people with prior system-design exposure who want to strengthen how they design, critique and guide agentic systems.

Explore the programme and bring a learning question to the application.

learning.thelivingcraft.ai

Image — Design agentic systems. Guide your team.

Build a working system. Explain your decisions. Develop your reasoning through continuous feedback.
30 live hours plus independent work.
Explore the open cohort.

Whether an agent is needed

Audience: Experienced engineer or technical lead · Practical question

Caption

Before choosing an agent framework, I would ask what actually needs an agent.

This illustrative planning example starts with a task that sounds like one job. Looking closely reveals several different jobs—and some are better handled by ordinary code.

Try the same separation on your next proposal: where is judgment needed, and where would a rule be enough?

Spoken script

Imagine your team is building a service that prepares a daily plan. Someone suggests an agent, and the conversation immediately turns to models and frameworks.

I would pause one step earlier. What decisions are we asking the system to make?

Checking whether a required field is present does not need a model. Neither does enforcing a fixed limit. Those are jobs we can describe and test directly.

Choosing between several reasonable options, with incomplete context, is a different problem. That is where I would examine whether a model adds useful judgment—and whether an agent needs tools to gather information or act.

I would also ask what happens when the answer is wrong. Can someone review it? Can we recover? How will we tell whether the result was useful?

In this illustrative example, the design might end up with a small agentic part surrounded by quite ordinary software. That is a reasonable outcome.

The question is whether the approach earns its complexity for this task.

Take one proposed agent in your team. Separate the fixed rules, the uncertain decisions and the permitted actions. Which part still needs an agent after you have done that?

Demonstrate a use-case review

Audience: Experienced engineer or technical lead · Try the review

Caption

A useful design review can make an agent smaller.

Consider an illustrative expense assistant. It reads a request, checks policy and prepares a recommendation. A separate service authorises any payment.

I would review each step before deciding where a model belongs. Fixed limits can be enforced in code. Unclear descriptions may need interpretation. Missing evidence needs a defined next step.

The result is a clearer allocation of responsibility, with fewer decisions left to an open-ended prompt.

Try the review questions in this example against one workflow you know.

Panel 1 — Start with the job

An expense assistant should help resolve a request. “Use an agent” does not explain the job.

Panel 2 — Separate the fixed rules

A spending limit can be checked in code. A model does not need to reinterpret it on every run.

Panel 3 — Find the uncertain decision

An unclear description may need interpretation or another question. Name what information would resolve it.

Panel 4 — Bound the action

Preparing a recommendation and authorising a payment carry different responsibilities. Define each permission.

Panel 5 — Decide how to check it

Use cases with known outcomes, review ambiguous cases and test what happens when evidence is missing.

Panel 6 — Review your own workflow

Which steps need interpretation? Which need tools? Which can stay deterministic?
Illustrative teaching example.

Model selection

Audience: Experienced engineer or technical lead · Practical question

Caption

“Which model should we use?” can hide several different decisions.

Imagine a system with a planner, a checker and a writer. The planner chooses a path. The checker examines constraints. The writer explains the result.

Giving them all the same model is convenient. It may also mean paying for capability a step does not need—or asking a model to do a check that code could perform more reliably.

I would compare the jobs before comparing the models. Then I would test candidate approaches on the work each role actually does.

A second model can offer another view, but a different vendor or model family does not by itself prove independent judgment.

For one role in your system, write down what would make you change your current choice.

Panel 1 — Choose for the job

A planner, checker and writer can have different needs. Start by naming the decision each one owns.

Panel 2 — Define acceptable behaviour

What must this role get right? Which failure would make the output unusable?

Panel 3 — Compare on your cases

Use representative inputs and the same acceptance criteria. Include difficult and incomplete cases.

Panel 4 — Count the surrounding cost

Include retries, review time and failed work alongside model charges. A cheaper call can cost more overall.

Panel 5 — Test the checker too

A second model can share mistakes. Compare its judgments with known cases and qualified human review.

Panel 6 — Make the choice revisitable

Record the reason, the limitation and the evidence that would trigger a new comparison.
Illustrative model-selection review.

Demonstrate a reasoned comparison

Audience: Experienced engineer or technical lead · Try the review

Caption

Suppose a cheaper model produces a fluent answer but repeatedly misses a required condition.

Would you choose it?

I would want to know whether that condition is essential, whether another part of the system enforces it, and what the missed check costs someone downstream.

This illustrative comparison is the kind of reasoning I want members to practise: make a choice, explain the evidence and stay open to revising it.

Use the questions here when a model comparison turns into an argument about a leaderboard.

Panel 1 — Compare a decision

Illustrative case: choose a model for drafting a service plan that must respect fixed constraints.

Panel 2 — Start with the constraint

One candidate writes clearly but misses a required condition. Decide whether that failure is acceptable before looking at price.

Panel 3 — Inspect the whole system

Can a deterministic check detect the omission? What happens after it is detected? Include the cost of that recovery.

Panel 4 — Compare like with like

Use the same representative cases and criteria. Record latency, successful work, failures and review effort.

Panel 5 — Explain the trade-off

“It is cheaper” leaves out the consequence. Explain what improves, what becomes harder and what remains uncertain.

Panel 6 — Say what would change your mind

A decision is easier to review when the next piece of evidence is explicit. What would you test next?

Architecture

Audience: Experienced engineer or technical lead · Apply

Caption

The architecture diagram looks complete. Can it explain a failed run?

In this illustrative example, I follow a planner, a checker and the part of the system that decides whether to retry. The boundaries become easier to discuss when we put an actual behaviour beside the boxes.

That connection between a working system and its design is central to The Living Craft.

Explore the open cohort: learning.thelivingcraft.ai

Spoken script

Imagine reviewing an agent architecture with a planner, some tools and a checker. The arrows form a neat loop.

Now suppose the checker rejects the result. What happens next?

If the answer is “the agent tries again,” I would ask what changes on that next attempt. Is there new information? Another valid option? Or will the system repeat the same failing step?

In this illustrative design, I would give the checker a structured result: which condition failed, what evidence it used and whether the workflow can recover.

Then I would make the retry decision explicit. The orchestrator needs a stopping rule and a path for cases it cannot resolve. A model should not have to invent those responsibilities while it is running.

Finally, I would ask for a trace that lets someone reconstruct the decision without exposing unnecessary personal data.

Now the diagram can explain a behaviour. We can build the loop, test a failure and revise the design when the evidence calls for it.

At The Living Craft, that is the connection I want members to practise: a working agentic system, the reasoning behind it and feedback that helps improve the next decision.

If you already work with system design, explore the open cohort and bring a question you would like to work through.

Show a design-review conversation

Audience: Experienced engineer or technical lead · Explore programme

Caption

What does feedback on an architecture actually sound like?

Here is an illustrative review. An agent keeps retrying after its checker rejects the output. The diagram contains all the expected components, but the next action is unclear.

I would start by asking what changes between attempts. That question can reveal a missing recovery path more quickly than adding another component to the diagram.

At The Living Craft, the working system gives feedback something concrete to refer to. Members explain their choices, examine the behaviour and revise the design.

Explore the programme: learning.thelivingcraft.ai

Panel 1 — A review starts with behaviour

Illustrative example: the checker rejects an output and the agent tries again. The same rejection repeats.

Panel 2 — Ask the next question

What new evidence or valid option does the next attempt have? If nothing changes, why would it succeed?

Panel 3 — Locate the decision

The checker identifies the failure. The workflow needs an explicit decision about recovery, stopping or human review.

Panel 4 — Make a change you can test

Bound the attempts. Define the unresolved outcome. Record enough evidence to explain why the workflow stopped.

Panel 5 — Run the difficult case again

Does it stop correctly? Can a reviewer understand the failure? What uncertainty remains?

Panel 6 — Practise the reasoning

Build, explain, examine and revise through continuous feedback.
Design agentic systems. Guide your team.
learning.thelivingcraft.ai